

DevOps Foundations & Continuous Integration

Task 3 – Automate Application Deployment

Student	Karan 2301410015
Platform	Ubuntu Linux + Jenkins + GitHub
Jenkins Job	Lab1-Basic-CICD

This report documents the actual Jenkins-based application deployment performed for Lab 1, including the configured build trigger, source checkout, Build/Test/Deploy pipeline, post-build action, and verification of the deployed application.

Submission evidence is based only on the screenshots captured during the completed experiment.

1. Objective & Deployment Workflow

The objective of Task 3 is to automate application deployment using Jenkins by configuring a build trigger, deployment steps, and a post-build action. The completed workflow uses GitHub as the source repository and Jenkins as the CI/CD automation server.

Implemented workflow



Requirement-to-evidence mapping

Requirement	Evidence captured
Build trigger	Jenkins Configure → Triggers → Poll SCM → H/5 * * * *
Deployment steps	Deploy stage removes/creates /tmp/lab1-deployed and copies app/index.html

Post-build action	Jenkins Declarative Post Actions reports deployment completion
Automation result	Jenkins Build #4 completes the pipeline successfully
Deployment verification	Deployed HTML is served locally and the browser displays the application

Key configuration

- Jenkins job: **Kunal-Lab1-Basic-CICD**
- Repository branch: **main**
- Trigger schedule: **H/5 * * * ***
- Deployment directory: **/tmp/lab1-deployed**
- Verification server: Python HTTP server on local port 8000

2. Build Trigger Configuration

Jenkins was configured with the Poll SCM trigger using the schedule H/5 * * * *. This instructs Jenkins to periodically check the configured source repository for changes and start a build when changes are detected.

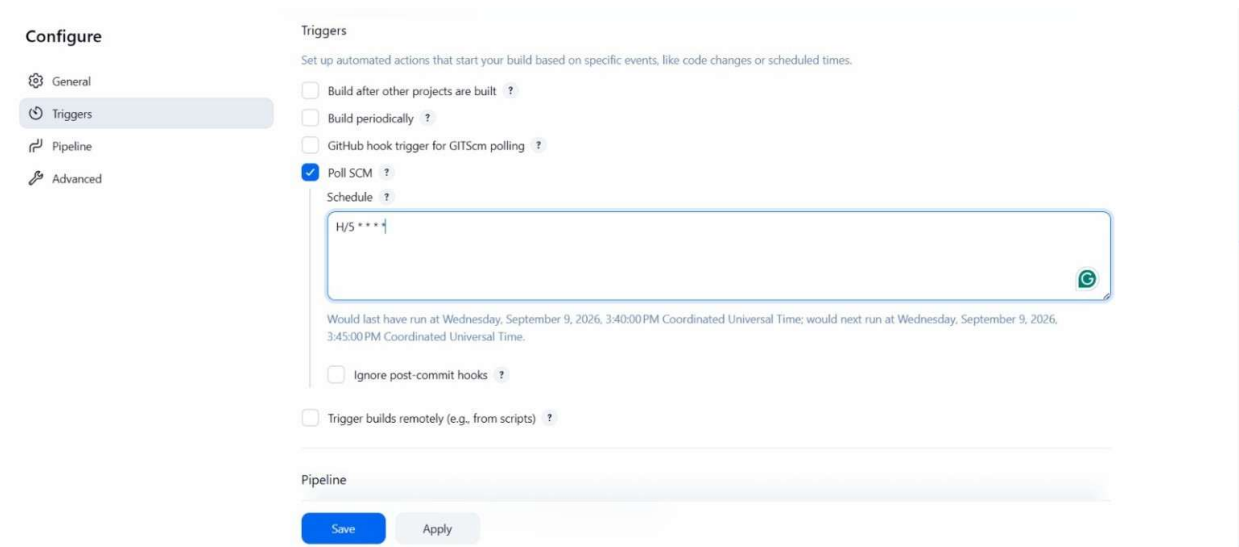


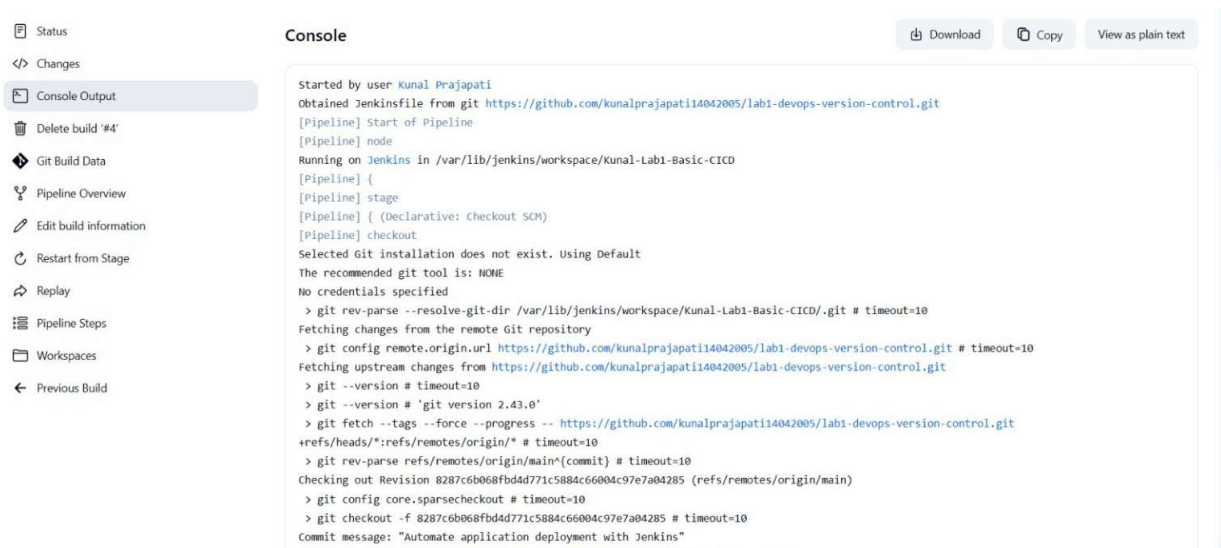
Figure 1. Jenkins Trigger configuration showing Poll SCM enabled with schedule H/5 * * * *.

Evidence interpretation

The screenshot directly shows the **Poll SCM** checkbox enabled and the configured schedule. This satisfies the build-trigger portion of Task 3.

3. Source Checkout & Pipeline Start

The Jenkins job obtains the Jenkinsfile from the GitHub repository and checks out the main branch before executing the pipeline. The console output identifies the repository and the revision used for Build #4.



The screenshot shows the Jenkins console output for Build #4. On the left, a sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#4', Git Build Data, Pipeline Overview, Edit build information, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main console area displays the following text:

```
Started by user Kunal Prajapati
Obtained Jenkinsfile from git https://github.com/kunalphrajapati14042005/lab1-devops-version-control.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/kunalphrajapati14042005/lab1-devops-version-control.git # timeout=10
Fetching upstream changes from https://github.com/kunalphrajapati14042005/lab1-devops-version-control.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/kunalphrajapati14042005/lab1-devops-version-control.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 8287c6b068fbd4d771c5884c66004c97e7a04285 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 8287c6b068fbd4d771c5884c66004c97e7a04285 # timeout=10
Commit message: "Automate application deployment with Jenkins"
```

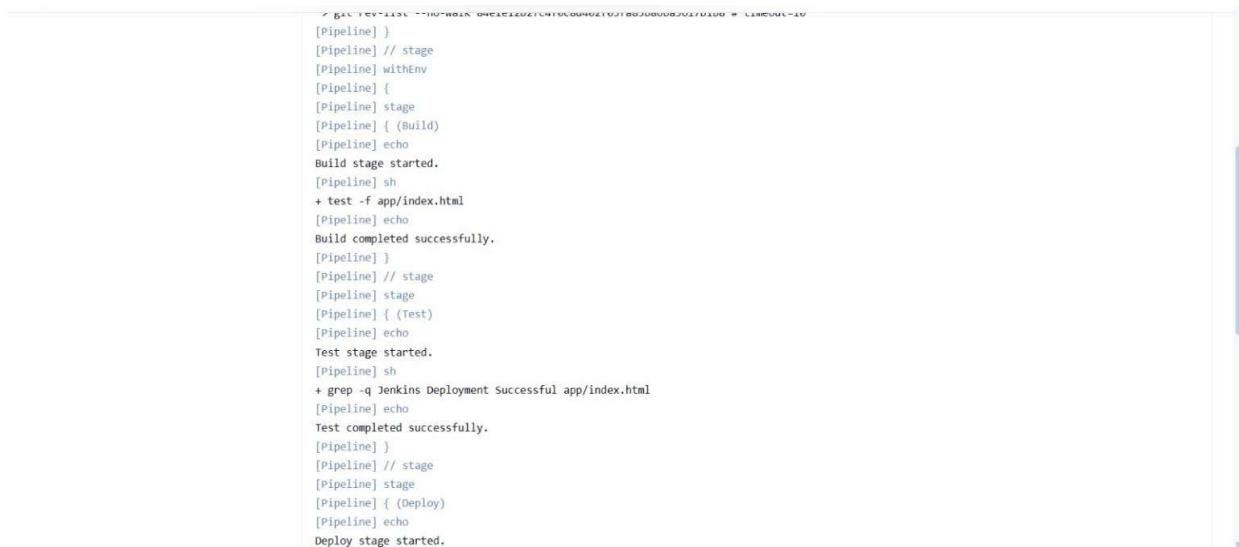
Figure 2. Jenkins Build #4 console showing GitHub repository checkout and pipeline initialization.

Observation

The source-controlled Jenkinsfile is retrieved from GitHub, allowing the deployment workflow to be executed automatically from the repository definition.

4. Build & Test Stages

After checkout, Jenkins executes the Build and Test stages. The Build stage verifies that `app/index.html` exists. The Test stage checks that the expected deployment text is present in the HTML file.



The screenshot shows the continuation of the Jenkins console output for Build #4. The main console area displays the following text:

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage started.
[Pipeline] sh
+ test -f app/index.html
[Pipeline] echo
Build completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage started.
[Pipeline] sh
+ grep -q Jenkins Deployment Successful app/index.html
[Pipeline] echo
Test completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage started.
```

Figure 3. Jenkins Build #4 console showing successful Build and Test stage execution.

Result

Both stages complete without failure, establishing that the application artifact is present and passes the defined content check before deployment.

5. Deployment Steps & Post-build Action

The Deploy stage performs the actual file deployment. Jenkins removes any previous deployment directory, creates /tmp/lab1-deployed, copies the application files into it, and lists the resulting deployment contents.

```
Deploy stage started.
[Pipeline] sh
+ rm -rf /tmp/lab1-deployed
[Pipeline] sh
+ mkdir -p /tmp/lab1-deployed
[Pipeline] sh
+ cp -r app/index.html /tmp/lab1-deployed/
[Pipeline] sh
+ ls -la /tmp/lab1-deployed
total 12
drwxr-xr-x  2 jenkins jenkins 4096 Sep  9 15:48 .
drwxr-xr-x 17 root    root    4096 Sep  9 15:48 ..
-rw-r--r--  1 jenkins jenkins 269 Sep  9 15:48 index.html
[Pipeline] echo
Application deployed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Post-build action: Deployment completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 4. Jenkins Build #4 console showing deployment to /tmp/lab1-deployed and the successful post-build action.

Deployment evidence

The console shows **index.html** present in /tmp/lab1-deployed and reports **Application deployed successfully**. The Declarative Post Actions section then reports **Post-build action: Deployment completed successfully**.

6. Deployment Verification & Conclusion

The deployed files were served from /tmp/lab1-deployed using Python's local HTTP server. A curl request returned the deployed HTML content, and the browser displayed the deployed application page.

Browser verification

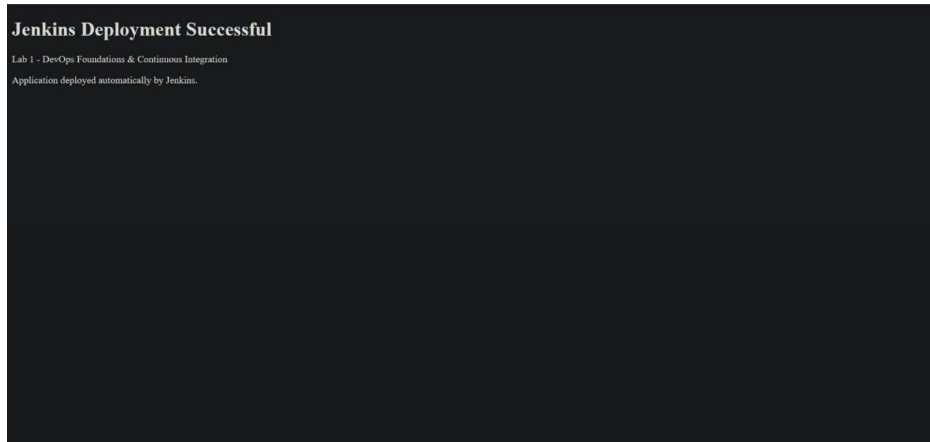


Figure 6. Browser displaying the deployed application with the message "Jenkins Deployment Successful".

Final conclusion

Task 3 was implemented using Jenkins automation. The workflow includes a configured Poll SCM build trigger, source checkout from GitHub, Build and Test stages, an actual deployment step to /tmp/lab1-deployed, a post-build action, and successful local application verification. The captured evidence demonstrates the complete deployment flow.